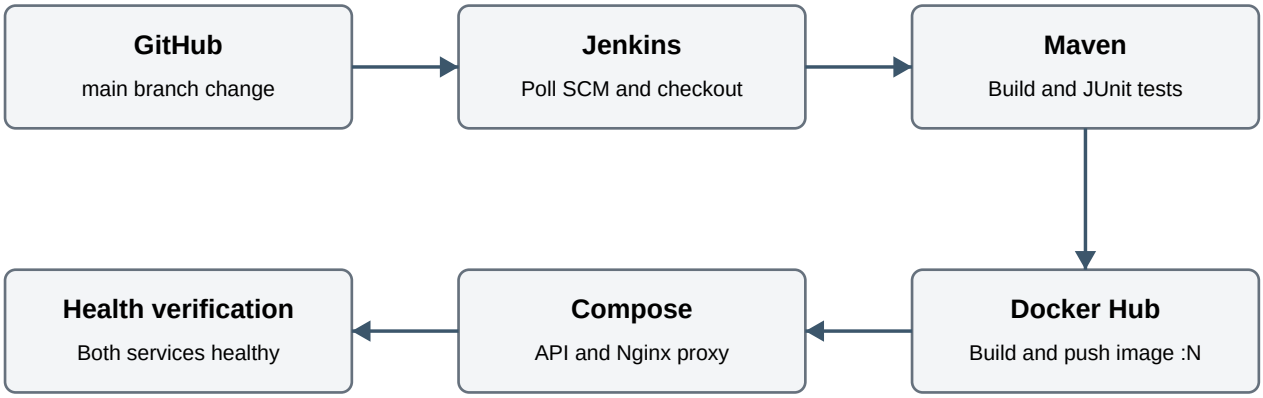
Calculator API CI CD Pipeline

CSE 816 Software Production Engineering
Mini Project Report | 30 October 2026 | ImNotFound7

Objective and scope

Build a small Spring Boot calculator service and automate the path from a GitHub change to a tested Docker Compose deployment. The implementation uses Java 17, Maven, Jenkins, Docker, and Nginx. Deployment stops at Compose, as required by the mini-project.

Pipeline architecture



Final build status -> Gmail SMTP notification

Application and runtime

GET endpoints under /api/v1 perform add, subtract, multiply, and divide operations using numeric a and b query parameters. Success responses include operation, inputs, and result. Invalid or missing inputs, division by zero, non-finite values, and overflow produce structured HTTP 400 problem details. /actuator/health reports service health.

Nginx publishes the stack at 127.0.0.1:8081 and forwards requests over the Compose network. The API container runs as a non-root user on internal port 8080. Both containers have health checks and restart policies. Arithmetic uses Java double precision, including its normal floating-point rounding limits.

Pipeline stages and validation

Stage	Implementation and failure behavior
Checkout	Fetch the repository revision from GitHub. Poll SCM checks for changes every two minutes after the initial job run.
Build	Run mvn clean package -DskipTests to compile and produce target/calculator.jar. A compilation or packaging error stops the pipeline.
Test	Run mvn test and publish target/surefire-reports/TEST-*.xml. A failing test prevents image publication and deployment.
Docker Build	Build the image from a Maven/JDK stage and a JRE runtime stage. Dependencies are cached before sources are copied. Tag the image with the Jenkins build number.
Push	Read the dockerhub credential from Jenkins, authenticate using password-stdin, and push imnotfound/calculator-devops:N. Remove the temporary Docker authentication directory afterward.
Deploy	Run docker compose up -d --no-build --wait with a 120-second health timeout, then request /actuator/health through Nginx. Queue concurrent builds to protect the shared deployment.
Post actions	Send the final build status and build URL through Gmail SMTP using Jenkins-managed credentials. Collect container diagnostics when the pipeline fails.

Verification performed

Automated tests: mvn clean verify passed all 31 tests: 14 service unit cases and 17 Spring MVC integration cases. Tests exercise every operation, division by positive and negative zero, missing and malformed parameters, NaN/infinity, overflow, unsupported routes, and the Actuator health response. The executable JAR was produced successfully.

Container checks: the standalone docker run image returned the expected addition response. Compose brought both services to healthy state. Live requests through Nginx verified all four operations, four invalid-request cases, and health. Repeating Compose deployment succeeded without breaking the running stack.

Jenkins evidence: build #3 was started automatically by an SCM change after a push to main. It completed all six required stages, published image tag :3, and deployed healthy containers. All 31 tests passed in Jenkins. The Jenkinsfile also passed declarative validation. The success notification was visible in the recipient Gmail inbox.

Challenges and project evidence

Java version mismatch

The interactive java command used Java 17, but Maven inherited JAVA_HOME pointing to Java 11. Builds explicitly selected Java 17, and Jenkins received named JDK17 and Maven3 tool installations. Checking mvn -version catches this mismatch before compilation.

Localhost and port conflicts

An existing application and Jenkins both used port 8080 through different local bindings. localhost reached the other service and returned HTTP 404. Jenkins links were corrected to 127.0.0.1:8080, and the calculator proxy was assigned the free port 8081. Existing applications were left running.

Deployment replacement and credentials

Replacing the API changes its container address. Nginx resolves the Compose service through Docker DNS with a short refresh interval so it can reach the replacement. Registry authentication uses a temporary directory. Docker Hub and SMTP credentials remain in Jenkins and are excluded from Git.

Email authentication

Gmail requires a dedicated app password for this SMTP configuration. The first saved credential was rejected with SMTP 535; the credential was corrected in Jenkins without placing it in source control. Delivery was confirmed in the recipient Gmail inbox.

Repository and image

GitHub: github.com/ImNotFound7/calculator-devops
Merged implementation: pull request 1
Public image: hub.docker.com/r/imnotfound/calculator-devops

The history contains separate requirements, application/test, and pipeline/container commits followed by a feature-branch merge. Author and committer use the configured ImNotFound7 identity. The repository is private; grant the evaluator access before review. Build output, secrets, assistant configuration, and planning notes are excluded.

Live demonstration

Keep Jenkins and Docker running. Push a meaningful change to main, watch SCM polling start the build, inspect the published JUnit results and build-number image tag, then call the API through port 8081. Saved screenshots document the pipeline, received notification, running containers, and example API responses. The classroom demonstration must still be performed live.